BombLab 实验报告

2026 年 10 月 30 日

实验结果与任务

本实验通过分析个人炸弹的 x86 64 汇编，恢复 challenge 到口令的计算过程，并在 defuser.c 中实现通用求解。正式六关与必做的 Honor 均已完成；本地公开测试通过 60/60 和 10/10，公开列表之外的 seed 43 也完整通关。上述结果是本地验证记录，不代表隐藏测试评分。

环境与工具

项目	配置或用途
系统	Windows 上的 WSL2 Ubuntu 22.04，Linux x86_64
工作目录	D:\ICS\Lab\Lab2 WSL 对应 /mnt/d/ICS/Lab/Lab2
编译与测试	GCC 11.4.0、GNU Make 4.3、Python 3.10.12
分析工具	GDB 12.1、GNU binutils 2.38
分析对象	正式炸弹 ./bomb；教学示例 ./tutor/bomb

分析方法

我先在教学关中练习断点、查看寄存器与单步执行，再分析正式关卡。对每关先确认输入整数个数，再沿参数传递找到计算函数；遇到间接跳转或指针，读取对应的跳转表、函数表或节点内存。根据这些观察恢复表达式，整理到求解接口，再用公开测试和额外输入核对。

第 1 关保留了实际单步调试的中间值。后续关卡的记录主要来自反汇编、表数据和节点数据的读取，以及交互核对后的自动验证；下文标为“代入复核”的数值由恢复的公式计算。分析、实现整理和验证使用了 AI 辅助，具体范围见末页。

阅读汇编时采用的约定

AT&T 语法按“源、目的”书写。函数入口的参数寄存器含义由调用者决定；在这些 phase 函数入口，rdi 是输入文本地址，rsi 是 challenge。r12 或 r13 保存 challenge 后，esi 可被复用来传递字段数。寄存器本身没有固定变量名。

下文以 c 表示 uint64_t challenge，U32(x) 表示取低 32 位。中间计算使用 uint32_t，按模 2^32 回绕；shr 为逻辑右移，^ 为按位异或。写 eax 等 32 位寄存器会清零对应 64 位寄存器的高半部分。lea 在这些计算中用于加法与倍数运算，并不读取该地址的内存。

第 1 关 函数调用与数值计算

phase_01 在 0x402090，要求一个无符号十进制整数。查看 0x403189 得到格式串 %u；解析后比较 sscanf 返回值是否为 1，再将输入整数与 p1_compute 的返回值比较。

关键汇编与数据流

```
# phase_01
mov %rsi,%r12
mov %r12,%rdi
call p1_compute
cmp %eax,-0x14(%rbp)
# p1_step0
lea 0x9079(%rsi),%eax
xor %edi,%eax
# p1_step1
xor $0x980a,%esi
lea (%rsi,%rdi,4),%eax
# p1_step2
lea (%rdi,%rdi,4),%eax
add %esi,%eax
xor $0x20a5,%eax
```

p1_compute 拆出 challenge 的低 32 位、高 32 位和右移 16 位后的低 32 位。第一个调用结果保存在 ecx；后两次调用完成后，再用 xor %ecx,%eax 合并结果。因此不能只取最后一次子函数的返回值。

恢复的计算关系

```
low  = U32(c);
high = U32(c >> 32);
mid  = U32(c >> 16);
step0 = (high + 0x9079u) ^ low;
step1 = (mid ^ 0x980au) + high * 4u;
step2 = (step1 * 5u + low) ^ 0x20a5u;
values[0] = step0 ^ step2;
```

实际单步观察

seed 42 的 challenge 为 0x28efe333b266f103。我在各 step 的入口和返回位置查看寄存器，并在最终 xor 后读取 eax，得到下表；随后输入十进制口令，原炸弹显示 Phase 1 defused.。

量	十六进制观察值
step0	0x9a9682af
step1	0x86f2b738
step2	0x5524a5be
最终结果	0xcfb22711 = 3484559121

公开列表之外的验证

```
seed = 43
challenge = 0x9cde98852e60034b
input = 3638523601
```

求解结果送入同 seed 的原炸弹，本关显示 defused。完整记录与截图见末两页。

第 2 关 数组与循环递推

phase_02 在 0x4020f0，要求六个无符号整数。六个输出地址从 rbp−0x40 到 rbp−0x2c 连续排列，每项 4 字节。读取成功后，第一项单独检查，其余五项在循环中逐项验证。

关键汇编

```
# 第一项
shr $0x18,%rax
and $0x1f,%eax
# 循环初值 ecx = 9，edx = 1
mov %r13,%rax
lea (%rsi,%rsi,2),%esi
shr %cl,%rax
sub %edx,%esi
and $0xf,%eax
add $0x1,%eax
xor %esi,%eax
mov -0x40(%rbp,%rdx,4),%esi
cmp %eax,%esi
add $0x1,%rdx
add $0x2,%ecx
```

r13 保存入口传入的 challenge；edx 是下标 i，ecx 是移位数 shift。每轮开始时 esi 中是前一项，lea 先乘 3，sub 再减去 i。eax 从 challenge 提取四位后加 1，再与该基数异或。验证当前项时读入的 esi，又在下一轮成为前一项。

恢复的计算关系

```
values[0] = U32((c >> 24) & 31u);
uint32_t shift = 9u;
for (uint32_t i = 1u; i < 6u; ++i) {
    uint32_t base = values[i - 1] * 3u - i;
    uint32_t extra = U32((c >> shift) & 15u) + 1u;
    values[i] = base ^ extra;
    shift += 2u;
}
```

代入复核

对 seed 43，第一项为 5。后五轮的 shift 依次为 9、11、13、15、17。base 的减法也必须按无符号 32 位执行，以保留可能发生的回绕。

i	shift	base	extra	values[i]
1	9	14	13	3
2	11	7	12	11
3	13	30	3	29
4	15	83	1	82
5	17	241	5	244

公开列表之外的验证

```
seed = 43
challenge = 0x6ec624aa85685867
input = 5 3 11 29 82 244
```

求解结果送入同 seed 的原炸弹，本关显示 defused。完整记录与截图见末两页。

第 3 关 switch 与跳转表

phase_03 在 0x402290，读取两个整数。第一项必须等于 challenge 的低三位；第二项等于分派函数的返回值。调用 p3_dispatch 时，edi 为选择值，esi 为待计算的 input。

```
selector = U32(c & 7u);
input = U32(c >> 3u) ^ U32(c >> 32u);
values[0] = selector;
```

关键汇编与表读取

```
# p3_dispatch，0x402230
mov %edi,%eax
mov %esi,%edi
cmp $0x7,%eax
ja 0x402285
notrack jmp *0x405820(,%rax,8)
# GDB 读取 8 个 64 位地址
x/8ga 0x405820
```

mov %esi,%edi 将 input 改放为子函数的第一个参数；eax 保留 selector，用作 8 字节表项的下标。读取表之后，还要沿表项地址处的 jmp 找到实际计算函数。不能按 p3_case 的名称后缀直接排列 switch。

选择值	跳转地址	实际函数	第二项计算式
0	0x402258	p3_case_6	input * 3u + 0x47c3u
1	0x402260	p3_case_3	(input * 2u) ^ 0x98acu
2	0x402268	p3_case_4	(input * 2u) ^ 0xc22u
3	0x402270	p3_case_1	input * 5u + 0x10414u
4	0x402278	p3_case_5	input * 5u + 0x13e16u
5	0x402280	p3_case_2	input * 3u + 0x483au
6	0x402248	p3_case_0	(input ^ 0xae1du) + (input >> 1u)
7	0x402250	p3_case_7	(input ^ 0x8e18u) + (input >> 1u)

分支内表达式的依据

```
# p3_case_5
lea 0x13e16(%rdi,%rdi,4),%eax
# p3_case_7
mov %edi,%eax
shr %edi
xor $0x8e18,%eax
add %edi,%eax
```

前者是 5 倍 input 加常数，lea 中的位移量表示加法；后者保留原 input 后将另一份右移一位，分别异或与相加。最终用 selector 的 switch 将对应结果写入 values[1]。

公开列表之外的验证

```
seed = 43
challenge = 0xd49ffe504bfdab7f
input = 7 1288760006
```

求解结果送入同 seed 的原炸弹，本关显示 defused。完整记录与截图见末两页。

额外直接 challenge 检查覆盖全部八个选择值，在三组高位模式下共 24/24 通过。

第 4 关 递归与返回值组合

phase_04 在 0x402380，只读取一个整数。入口准备 target、区间下界 0、上界 127 和 mix，按 rdi、rsi、rdx、rcx 的顺序传给 p4_walk。口令就是这次递归调用的 32 位返回值。

```
target = U32(c & 127u);
mix = U32(c >> 7u) ^ U32(c >> 32u);
values[0] = p4_walk(target, 0u, 127u, mix);
```

关键汇编

```
# 计算 mid 并检查是否命中
mov %edi,%eax
mov %edx,%edi
sub %esi,%edi
shr %edi
add %esi,%edi
cmp %eax,%edi
je 0x402370
# 保存 mid + mix，判断 target < mid
lea (%rdi,%rcx,1),%ebx
cmp %edi,%eax
jb 0x402350
# 返回路径中：5 * child，再与 mid + mix 异或
lea (%rax,%rax,4),%eax
xor %ebx,%eax
```

eax 先保存 target，edi 改作 mid。cmp %edi,%eax 后的 jb 表示无符号 target < mid。左支将上界改为 mid−1，右支将下界改为 mid+1。ebx 保存当前层的 mid+mix，跨递归调用后参与返回值计算；左右两支最后分别异或 10 和 7。

恢复的递归关系

```
mid = low + ((high - low) >> 1u);
if (target == mid)
    return (mid + 0x48u) ^ mix;
if (target < mid) {
    child = p4_walk(target, low, mid - 1u, mix);
    return (child * 5u) ^ (mid + mix) ^ 10u;
}
child = p4_walk(target, mid + 1u, high, mix);
return (child * 5u) ^ (mid + mix) ^ 7u;
```

终止条件与代入复核

target 被限制在 0～127，初始区间包含它，每次未命中都缩小区间，因此能够命中。seed 43 的 target 为 127，mix 为 0x368a5338；各层 mid 依次为 63、95、111、119、123、125、126、127。到达基例后，逐层按右支公式组合，得到口令。

公开列表之外的验证

```
seed = 43
challenge = 0x006ff81b72d591ff
input = 2467722972
```

求解结果送入同 seed 的原炸弹，本关显示 defused。完整记录与截图见末两页。

为覆盖所有递归目标，另用三组 challenge 高位模式分别枚举低七位 0～127，共 384/384 通过。

第 5 关 结构体与链表遍历

phase_05 在 0x402460，读取两个整数。p5_traverse 的返回值与第一项比较；通过第二个指针参数写回的 tag 与第二项比较。原节点布局为 value 占 4 字节、tag 占 4 字节、next 指针占 8 字节，总计 16 字节。

关键指令与节点恢复

```
mov 0x405b60(,%rax,8),%rdx  # 选头指针
mov 0x4(%rdx),%esi          # tag
add (%rdx),%eax             # value
mov 0x8(%rdx),%rdx          # next
# 实际使用的 GDB 查看命令
x/8ga 0x405b60
x/24gx 0x405ba0
```

节点基址为 0x405ba0。小端序下，每节点第一个 64 位数的低 32 位是 value，高 32 位是 tag；第二个 64 位数是 next 地址。按 (地址−基址)/16 转为节点下标，得到下表。头表顺序为 [4, 0, 3, 7, 11, 8, 9, 2]。

节点下标	value 十六进制	tag 十六进制	next 下标
0	bcd3	441	3
1	c7f1	33	5
2	aba1	56c	1
3	c772	a88	7
4	fd52	60b	0
5	464a	778	6
6	19fd	b63	10
7	ca61	227	11
8	2acf	359	9
9	d968	841	2
10	14c2	10a	NULL
11	101a	944	8

恢复的遍历关系

```
start = U32(c & 7u);
hops = U32((c >> 3u) & 3u);
acc = U32(c >> 32u) ^ U32(c >> 6u) ^ 0x2e28u;
node = heads[start];
// 先处理头节点，再沿 next 移动 hops 次
for (step = 0u; step <= hops; ++step) {
    if (step > 0u) node = nodes[node].next;
    acc = (acc * 3u + nodes[node].value) ^ nodes[node].tag;
    last_tag = nodes[node].tag;
}
values[0] = acc; values[1] = last_tag;
```

hops 计数的是沿 next 移动的次数，所以处理节点数为 hops+1，即 1～4 个。实现用下标重建链表关系。seed 43 的 start=6、hops=3，路径为 9→2→1→5；acc 依次为 3155794387、877493366、2632531296、3602643218，末节点 tag 为 1912。

```
seed 43: 0x18e47d0995aecbfe
input: 3602643218 1912    result: Phase 5 defused.
```

另对八个头节点、四种 hops 和三组高位模式组合验证，共 96/96 通过。

第 6 关 函数指针与两次间接调用

phase_06 在 0x4025b0，读取一个整数并与 p6_dispatch 返回值比较。该函数不是连续执行六种运算，而是从同一个函数指针表选出两个运算，前一次结果经过异或后作为后一次输入。

关键汇编与指针表

```
# rbx 原先保存 challenge
and $0x38,%ebx
and $0x7,%eax
call *0x405e00(,%rax,8)
xor $0xd8a8,%eax
mov %eax,%edi
call *0x405e00(%rbx)
add $0xcb03,%eax
# GDB 读取函数指针表
x/8ga 0x405e00
```

第一处间接调用显式将下标乘 8；第二处使用的 rbx 已经是字节偏移 c & 0x38，因此换算成下标是 (c >> 3) & 7。表中存在重复函数，必须保留其真实顺序。

```
ops = [op0, op2, op2, op3, op5, op4, op4, op1];
```

函数	恢复的 uint32_t 表达式
op0	((x << 5u) + 0x2548u) ^ (x * 3u)
op1	((x >> 3u) ^ x) + 0x6e01u
op2	(x + 0xa91cu) ^ (x >> 3u)
op3	((x << 2u) | (x >> 30u)) ^ 0xcb4fu
op4	(x ^ 0xf1bdu) + (x << 7u)
op5	(x >> 2u) | (x << 30u)

op3 的 rol $0x2,%eax 为 32 位循环左移两位；op5 的 ror $0x2,%eax 为循环右移两位。移出的位要回到另一端，因此使用两次移位再按位或，不能只用普通移位。

恢复的调用关系

```
input = U32(c >> 6u) ^ U32(c >> 32u);
first = U32(c & 7u);
second = U32((c >> 3u) & 7u);
result = ops[first](input);
result ^= 0xd8a8u;
values[0] = ops[second](result) + 0xcb03u;
```

代入复核与验证

seed 43 的 first=0、second=1，对应 op0 后接 op2。input=0x274bec67，第一次返回 0x9c9e771d，异或后为 0x9c9eafb5，第二次返回 0x8f0c8d27，最终加上常数得到 2400016426。

```
seed 43: 0x523fb4dd5d162e88
input: 2400016426    result: Phase 6 defused.
```

另对全部 64 对下标组合与三组输入高位模式验证，共 192/192 通过。

Honor 输入布局与上下文计算

Honor 的 secret_gate 在 0x402710。分析调用者可知，其参数依次为输入文本、Honor challenge 和 context。驱动通过 transcript_context 得到前六关的上下文；不能只根据 Honor challenge 求出口令。

入口布局与关键汇编

```
lea -0xd0(%rbp),%r12        # 复制目的地址
call secret_copy
cmpl $0x4e45504f,-0xb0(%rbp)
jne 0x40275b
lea -0xac(%rbp),%rdi        # 数字文本起点
call secret_phase
```

secret_copy 从源地址逐字节复制，遇到空字节才停止，并复制该空字节。检查字段相对复制起点的偏移是 0x20，即 32；数字文本从偏移 36 开始。0x4e45504f 在小端内存中的四个字节为 4f 50 45 4e，表示 OPEN。入口的 strlen 检查要求文本长度为 36～163 字节。

偏移	内容
0 至 31	32 个 A 作为填充
32 至 35	OPEN
36 起	三个无符号十进制整数，以空格分隔

三个整数的恢复关系

secret_phase 通过 %u %u %u 读取三项。r12 保存 challenge，r13 保存 context。secret_mix 中的关键指令为 shr $0x3,%eax、xor $0x9aa7,%eax、xor $0x7129,%edi 和 add %edi,%eax，因此右移位数是 3。

```
// k 为 uint64_t context，以下中间量均为 uint32_t
first = U32(k & 3u);
cfold = U32(c) ^ U32(c >> 32u);
kfold = U32(k) ^ U32(k >> 32u);
adjust = (first * 0x7129u) ^ U32(k >> 16u);
secret_mix(x) = ((x >> 3u) ^ 0x9aa7u) + (x ^ 0x7129u);
second = secret_mix(cfold ^ kfold) ^ adjust;
sum = kfold + first + 0x9aa7u + (cfold >> 3u);
third = second ^ sum;
```

使用 snprintf 将 32 个 A、OPEN 和三个数字写入 out，并检查出错或截断；由驱动添加行尾换行。此布局满足实际检查位置，不需要改写返回地址，也未修改炸弹数据来绕过验证。

公开列表之外的验证

```
seed = 43
challenge = 0xe22a53d064a0b63a
context   = 0x15036f7bb9fa01f9
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN1 1085877976 4252126015
result: Secret phase defused.
```

在同一 seed 下先输出前六关，再追加 Honor 第七行交给原炸弹。额外 15 个非公开 seed 均通过，context 低两位覆盖 0、1、2、3。seed 43 的入口参数也经过只读 GDB 观察核对。

验证记录与结果分析

公开列表包含 seed 0、1、7、42、99、123、2026、65535、987654321、18446744073709551615。选取其中没有的 43 进行完整运行，六个正式关卡与 Honor 全部显示 defused。各项输入来自求解器，未将这些实例数值写死到算法中。

seed 43 的完整记录

关卡	challenge	生成的输入
1	`0x9cde98852e60034b`	`3638523601`
2	`0x6ec624aa85685867`	`5 3 11 29 82 244`
3	`0xd49ffe504bfdab7f`	`7 1288760006`
4	`0x006ff81b72d591ff`	`2467722972`
5	`0x18e47d0995aecbfe`	`3602643218 1912`
6	`0x523fb4dd5d162e88`	`2400016426`
Honor	`0xe22a53d064a0b63a`	`32 个 A 与 OPEN 后接` `1 1085877976 4252126015`

测试范围

检查	覆盖范围	结果
模板公开测试	正式六关 6 × 10；Honor 10	60/60；10/10
第 3 关额外检查	八个 selector × 三组高位模式	24/24
第 4 关额外检查	128 个 target × 三组高位模式	384/384
第 5 关额外检查	八个 start × 四种 hops × 三组模式	96/96
第 6 关额外检查	64 对调用下标 × 三组模式	192/192
Honor 额外 seed	15 个，覆盖四种 context 低两位	15/15

额外 Honor seed 为 43、2、3、4、5、6、8、9、10、11、12、13、14、15、16。额外直接 challenge 检查由辅助脚本将求解结果交给对应关卡验证，不替换模板测试。所有有限测试只能检验所覆盖输入；通用性仍依赖计算关系、位宽和数据表恢复正确。

复现命令

```
make test
# 分别生成前六行与 Honor 第七行，再完整运行
{ ./defuser --seed 43; ./defuser --secret --seed 43; } | \
  ./bomb --seed 43
```

运行截图与资料说明

以下两图为在 WSL 终端实际运行后截图。第一图保留测试汇总与 Honor 测试部分；第二图是 seed 43 的完整通关摘要。第二图使用 grep 筛选 challenge、defused 与 BOOM 行，红色为终端匹配高亮。

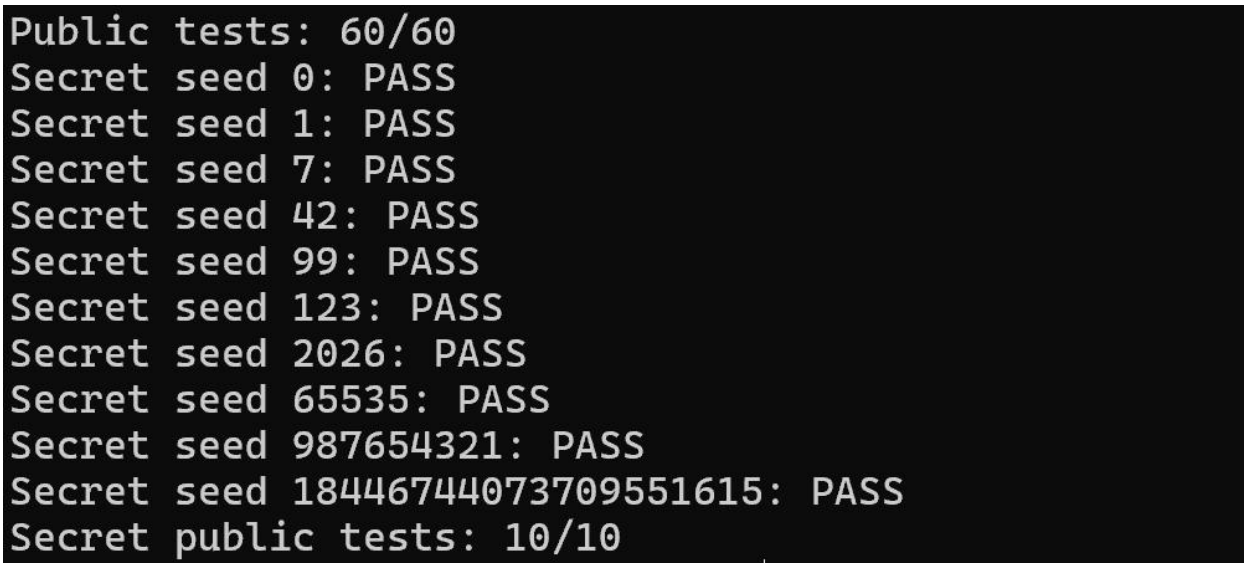


图 1 公开测试结果

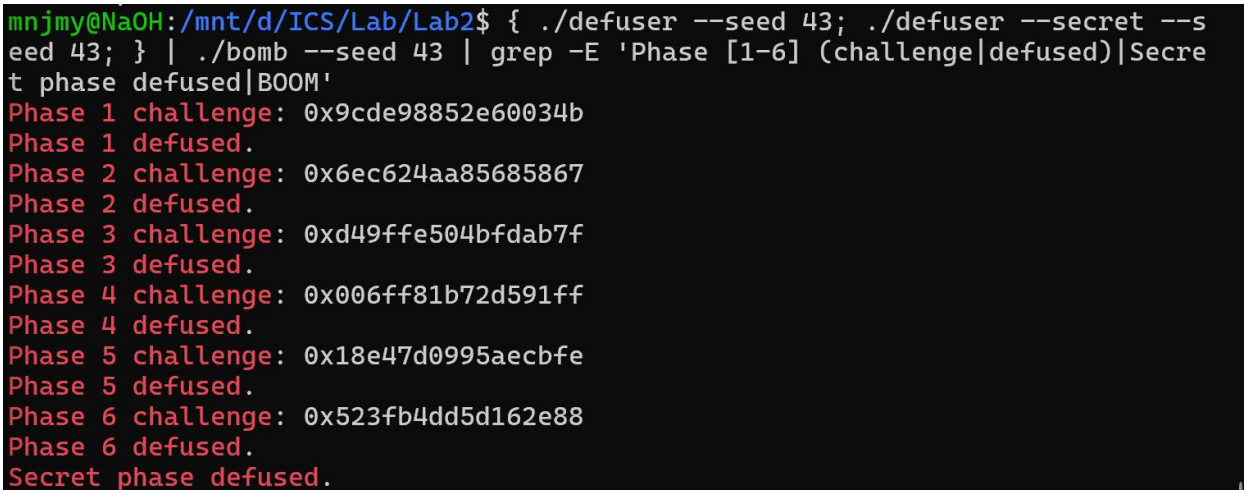


图 2 seed 43 完整通关结果 六关 challenge 与通关提示及 Honor 通关提示

参考资料

1. 课程 BombLab 实验说明：
https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/

2. ICS 课程官方 BombLab 模板：
https://github.com/ICS-26Fall-FDU/BombLab
参考仓库 README.md、tutor 说明、defuser.h、challenge.c 和 tests/public.json；分析对象为本人领取的 bomb。

辅助工具使用说明

使用 GDB 查看汇编、寄存器、格式串和内存表，使用 GCC、Make 与 Python 编译和验证。分析过程中使用 Codex 解释汇编和调用约定、提出调试步骤、核对我逐项写出的表达式并纠错；AI 还协助把已经讨论核对的逻辑整理进模板、执行自动测试、记录实验笔记并整理本报告。报告中的验证结果均对应实际运行记录，未将尚未执行的测试写为通过。